BOOTCAMP AI WEB PROGRAMMING · MODUL BACAAN SESI 9 · MODUL 1 DARI 2

Bekerja dalam Tim dengan GitHub

Cara tim pengembang membangun satu aplikasi tanpa saling menimpa pekerjaan

Bagian pertama membahas cara tim pengembang bekerja bersama: kolaborasi, *branch*, *pull request*, *code review*, *merge conflict*, komunikasi tim, dan *pair testing*. Bagian kedua menerapkannya pada aplikasi Bengkel Cemerlang Jaya Motor yang kamu bangun bersama tim.



Ioda Academy · Oktober 2026

Daftar Isi

Bagian A · Cara Tim Pengembang Bekerja

Bab 1 · Apa Itu Kolaborasi dalam Tim Pengembang?

1.1 Apa Itu *Collaboration*?
1.2 Masalah yang Muncul Tanpa Aturan Kerja
1.3 Bagaimana Tim Membagi Peran?
1.4 Alur Kerja Tim di GitHub

Bab 2 · *Branch* dan *Pull Request*

2.1 Apa Itu *Branch*?
2.2 Cara Melihat dan Membuat *Branch* di GitHub
2.3 Apa Itu *Commit*, *Push*, dan *Pull*?
2.4 Apa Itu *Pull Request*?
2.5 Judul dan Deskripsi *Pull Request* yang Baik

Bab 3 · *Code Review* dan *Merge*

3.1 Apa Itu *Code Review*?
3.2 Cara Meminta *Reviewer*
3.3 Cara Membaca Tab Files changed
3.4 Contoh Komentar *Review* yang Membantu
3.5 Cara Memberi Approve atau Request Changes
3.6 Apa Itu *Merge*?

Bab 4 · Apa Itu *Merge Conflict*?

4.1 Kapan *Merge Conflict* Terjadi?
4.2 Seperti Apa Penanda Konflik di File?
4.3 Tampilan Konflik di Halaman *Pull Request*
4.4 Cara Mencegah *Merge Conflict*

Bab 5 · Komunikasi dan Pengujian dalam Tim

5.1 Kapan Anggota Tim Harus Memberi Kabar?
5.2 Apa Itu Cek Singkat Harian?
5.3 Apa Itu *Pair Testing*?
5.4 Cara Menulis Laporan Temuan
5.5 Cara Menanggapi Temuan

Bagian B · Membangun App 4 Bersama Tim

Bab 6 · Menyiapkan Repo Tim Bengkel

6.1 Apa Isi Repo *Template* App 4?
6.2 Cara Ketua Tim Membuat Repo dari *Template*
6.3 Cara Mengundang Anggota sebagai *Collaborator*
6.4 Cara Anggota Tim Menerima Undangan
6.5 Pembagian Fitur A sampai D

Bab 7 · Mengerjakan dan Menggabungkan Fitur Bengkel

7.1 Cara Membuat *Branch* Fitur lewat Agen
7.2 File Mana yang Boleh Diubah?
7.3 Aturan Database Bersama
7.4 Cara Mengirim Fitur dengan Prompt *Push*
7.5 Cara Mencoba Fitur Teman dengan Prompt Coba
7.6 Empat Hal yang Diperiksa *Reviewer*
7.7 *Merge* oleh Ketua dan Prompt Tarik Main
7.8 Contoh Konflik di `src/lib/utils.ts`
7.9 Konflik di `src/types/database.ts`

Bab 8 · Kabar, Cek Singkat, dan Uji Silang Antartim

8.1 Tiga Saat Wajib Memberi Kabar di Kelas
8.2 Cek Singkat Pukul 11.30 dan 13.45
8.3 Skenario Uji Silang Antartim
8.4 Contoh Laporan Temuan Aplikasi Bengkel

Lampiran

A Daftar Istilah
B Daftar Pustaka

BAGIAN A · CARA TIM PENGEMBANG BEKERJA

BAB 1

Apa Itu Kolaborasi dalam Tim Pengembang?

Hampir semua aplikasi yang kamu pakai setiap hari dibangun oleh tim, bukan satu orang. Aplikasi toko online punya orang yang mengurus halaman produk, keranjang, pembayaran, dan laporan penjual. Mereka bekerja di satu kumpulan kode yang sama dan menyimpan data di database yang sama. Agar hasilnya utuh, tim butuh cara kerja yang disepakati.

1.1 Apa Itu *Collaboration*?

> **DEFINISI**
> ***Collaboration* (kolaborasi)** adalah kemampuan bekerja bersama orang lain menuju satu hasil: berbagi informasi tepat waktu, membagi tanggung jawab dengan jelas, saling memeriksa pekerjaan, dan menerima masukan tanpa merasa diserang.

Kolaborasi termasuk *soft skill*, yaitu kemampuan kerja yang berhubungan dengan cara berinteraksi, bukan dengan alat atau bahasa pemrograman tertentu. Di tim pengembang, kolaborasi terlihat dari hal-hal kecil: pengembang yang mengumumkan sebelum mengubah database bersama, penguji yang menulis laporan *bug* lengkap, atau pemilik fitur yang berterima kasih atas temuan. Kebiasaan kecil ini menentukan apakah beberapa orang yang pandai bisa menghasilkan satu aplikasi yang utuh.

Di tim pengembang, kolaborasi dijalankan lewat dua hal yang saling melengkapi: **alat** yang mengatur cara kode bertemu, seperti Git dan GitHub, serta **kebiasaan** antarmanusia, seperti memberi kabar, saling menguji, dan menanggapi temuan. Bab 2 sampai 4 membahas alatnya, Bab 5 membahas kebiasaannya.

1.2 Masalah yang Muncul Tanpa Aturan Kerja

Saat bekerja sendirian, hanya ada satu salinan kode yang berubah. Saat bekerja berempat, ada empat salinan yang berubah bersamaan, ditambah satu salinan di GitHub yang menjadi acuan bersama. Database biasanya hanya satu dan dipakai semua orang. Tanpa aturan, tiga masalah hampir selalu muncul.

Masalah	Contoh di tim aplikasi lain
Kode saling menimpa	Di tim toko online, pengembang A menambah pilihan kurir di halaman *checkout*, pengembang B menambah kolom kode promo di halaman yang sama. B menyalin file versinya setelah A. Kolom promo muncul, pilihan kurir hilang, dan tidak ada yang sadar sampai pembeli mengeluh.
Database berubah tanpa kabar	Di tim *mobile banking*, satu pengembang mengganti nama kolom `saldo` menjadi `saldo_akhir`. Halaman mutasi rekening buatan rekannya masih membaca `saldo` dan langsung menampilkan *error*. Rekannya mencari kesalahan di kodenya sendiri, padahal penyebabnya di database.
Fitur tidak tersambung	Di aplikasi tiket konser, tim pembayaran menyimpan status pesanan `paid`, sementara tim tiket menunggu status `lunas` sebelum mengirim e-tiket. Uang masuk, tetapi tiket tidak pernah terkirim. Masing-masing tim yakin bagiannya benar.

Kode bisa dipisah per orang dengan *branch* dan digabung dengan teratur lewat *pull request*. Database tidak bisa dipisah dengan cara itu, jadi yang menjaganya adalah aturan bersama dan kabar sebelum mengubah. Nama-nama yang dipakai lintas fitur dijaga dengan satu dokumen acuan, misalnya PRD, dan dengan pengujian alur lengkap setelah semua fitur digabung.

1.3 Bagaimana Tim Membagi Peran?

Tim kecil biasanya membagi pekerjaan per fitur. Setiap fitur punya satu pemilik yang bertanggung jawab sampai fitur itu masuk ke versi utama. Selain pemilik fitur, ada dua peran lain yang hampir selalu ada.

Peran	Tugas	Contoh di tim toko online
Pemilik fitur	Membangun satu fitur di jalur kerjanya sendiri, mengujinya, lalu mengajukannya untuk digabung.	Ani memegang keranjang, Budi memegang kode promo.
Reviewer	Memeriksa dan mencoba pekerjaan teman sebelum digabung.	Citra mereview fitur promo milik Budi.
Pengelola repo (ketua tim)	Menyiapkan repo, mengatur akses, menggabungkan pekerjaan yang sudah disetujui, dan menayangkan aplikasi.	Fajar menggabungkan fitur dan merilis aplikasi.

Satu orang bisa memegang lebih dari satu peran. Pengelola repo biasanya tetap memiliki satu fitur, dan setiap anggota bergantian menjadi *reviewer* bagi temannya. Yang penting, setiap peran jelas siapa pemegangnya.

1.4 Alur Kerja Tim di GitHub

GitHub menyebut alur kerja yang umum dipakai tim sebagai GitHub *flow*. Bagan berikut merangkum urutannya. Setiap langkah dibahas di Bab 2 dan Bab 3.



Inti alur ini sederhana: tidak ada yang mengubah versi utama secara langsung. Semua perubahan lewat jalur sendiri, diperiksa orang lain, lalu digabung oleh satu orang yang bertanggung jawab.

BAB 2

Branch dan *Pull Request*

Supaya perubahan satu orang tidak mengganggu yang lain, setiap anggota tim bekerja di jalur kode tersendiri. Setelah selesai, pekerjaan itu diajukan untuk digabung lewat halaman khusus di GitHub.

2.1 Apa Itu *Branch*?

DEFINISI

Branch adalah jalur kerja terpisah di dalam satu repo. Perubahan yang di-*commit* di sebuah *branch* tidak memengaruhi *branch* lain sampai sengaja digabungkan.

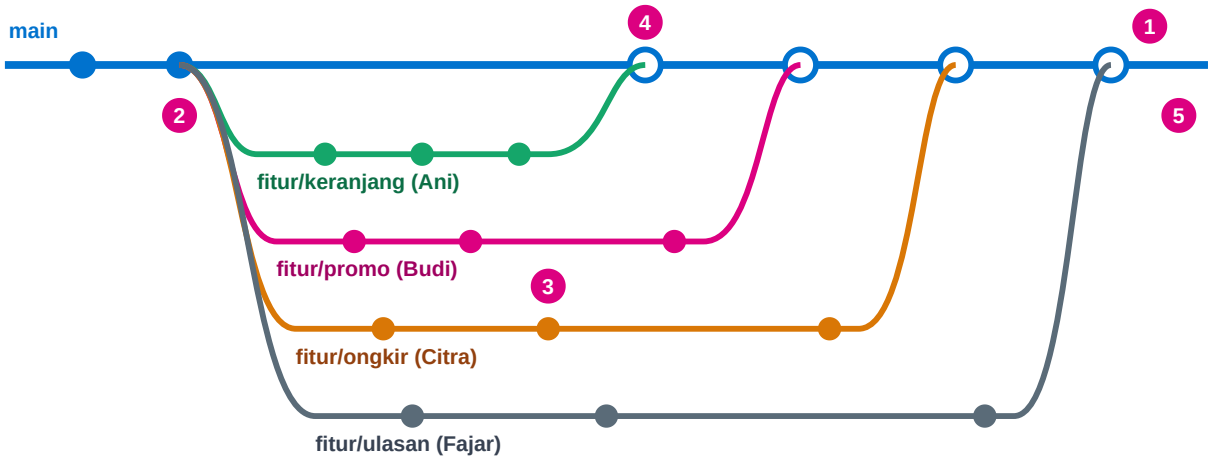
DEFINISI

main adalah *branch* utama repo, yaitu *branch* yang ditampilkan GitHub saat repo dibuka. Biasanya main berisi versi aplikasi yang sudah disetujui dan siap ditayangkan.

Dokumentasi GitHub menjelaskan bahwa *branch* dipakai untuk mengembangkan fitur, memperbaiki *bug*, atau mencoba ide baru di area terpisah, tanpa terganggu perubahan orang lain. Aplikasi yang dipakai pengguna tetap

berasal dari main, jadi fitur yang belum selesai tidak ikut tampil. Saat dibuat, isi *branch* sama dengan *branch* asalnya, karena itu *branch* fitur dibuat dari main terbaru.

Bagan berikut memperlihatkan main dan empat *branch* fitur di tim toko online sepanjang satu hari. Setiap lingkaran adalah satu *commit*.

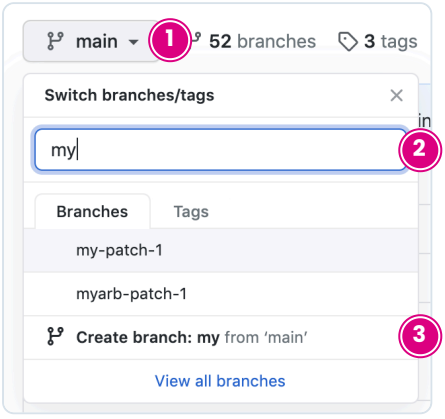


main (biru) dan empat branch fitur. Lingkaran kosong di main adalah titik pull request digabungkan.

1. **main.** Versi bersama yang sudah disetujui. Tidak ada yang mengubahnya secara langsung.
2. **Titik awal *branch*.** Keempat *branch* dibuat dari main terbaru.
3. ***Commit* di *branch* fitur.** Setiap anggota menyimpan kemajuannya di jalurnya sendiri. *Commit* Citra tidak terlihat di laptop Budi.
4. **Titik *merge*.** Fitur keranjang disetujui dan digabung ke main.
5. **Main bertambah.** Setiap kali satu fitur masuk, main berubah. Anggota lain menarik main terbaru ke *branch*-nya.

2.2 Cara Melihat dan Membuat *Branch* di GitHub

Di halaman repo, daftar *branch* dibuka lewat tombol bernama *branch* aktif di kiri atas daftar file. Tangkapan layar berikut memperlihatkan menu yang muncul.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Tombol *branch* aktif.** Menampilkan *branch* yang sedang dilihat, di sini `main`. Klik untuk membuka daftar.
2. **Kolom pencarian.** Ketik nama *branch* untuk mencarinya, misalnya `fitur/promo`. Pilih nama di daftar untuk melihat isi *branch* itu.
3. **Create branch.** Bila nama yang diketik belum ada, GitHub menawarkan membuat *branch* baru dari *branch* yang sedang dilihat.

Dalam praktik, *branch* lebih sering dibuat di laptop, lewat perintah Git atau agen AI, lalu dikirim ke GitHub. Menu ini tetap berguna untuk memeriksa *branch* apa saja yang sudah ada di repo tim.

2.3 Apa Itu *Commit*, *Push*, dan *Pull*?

Tiga istilah ini muncul di setiap alur kerja tim. ***Commit*** adalah simpanan keadaan kode pada satu titik waktu beserta pesan yang menjelaskannya. ***Push*** adalah mengirim *commit* dari laptop ke repo di GitHub. ***Pull*** adalah kebalikannya: mengambil *commit* terbaru dari GitHub lalu menggabungkannya ke *branch* di laptop.

Pesan *commit* yang baik menyebut apa yang berubah dengan bahasa yang bisa dipahami teman. Bandingkan dua contoh dari tim toko online berikut.

KURANG JELAS

`update` `fix` `coba lagi`

JELAS

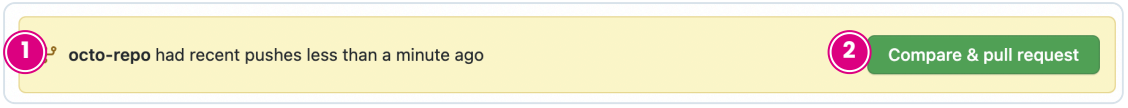
`Tambah kolom kode promo di checkout`
`Tolak kode promo yang sudah kedaluwarsa`

2.4 Apa Itu *Pull Request*?

DEFINISI

Pull request (PR) adalah usulan untuk menggabungkan perubahan dari satu *branch* ke *branch* lain, biasanya ke main. Halaman PR menampilkan deskripsi usulan, daftar *commit*, file yang berubah, serta komentar dan keputusan *reviewer*.

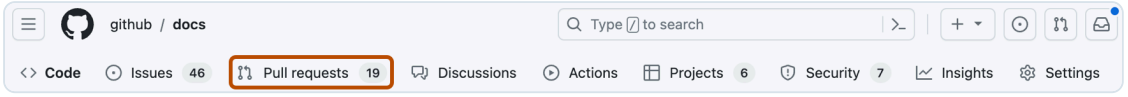
GitHub menyebut PR sebagai fitur kolaborasi utamanya, karena perubahan dibahas dan diperiksa di sana sebelum digabung. Setelah sebuah *branch* di-*push*, GitHub menampilkan pita kuning di halaman repo selama beberapa saat. Tangkapan layar berikut memperlihatkan pita itu.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Nama *branch* dan waktu *push*.** Menunjukkan *branch* yang baru saja dikirim.
2. **Compare & pull request.** Membuka formulir PR. Isi judul dan deskripsi, lalu klik **Create pull request**.

Bila pita sudah hilang, buka tab **Pull requests** di bawah nama repo, seperti yang diberi kotak oranye pada tangkapan layar berikut, lalu buat PR baru. Di formulir PR, pastikan arah penggabungan benar: *base* adalah main, sedangkan *compare* adalah *branch* fiturmu.



Sumber gambar: GitHub Docs (CC BY 4.0)

2.5 Judul dan Deskripsi *Pull Request* yang Baik

Judul dan deskripsi adalah hal pertama yang dibaca *reviewer*. Deskripsi yang baik menjawab tiga pertanyaan: apa yang dibuat, bagaimana cara mengujinya, dan apa yang perlu diperhatikan. Bandingkan dua contoh PR fitur kode promo di tim toko online.

KURANG MEMBANTU

Judul: update promo
Deskripsi: sudah jadi, tolong dicek ya

MEMBANTU

Judul: Kolom kode promo di halaman *checkout*
Yang dibuat: kolom kode promo dan potongan harga otomatis.
Cara menguji: masukkan `HEMAT10`, total turun 10%. Masukkan kode kedaluwarsa, muncul pesan ditolak.
Perlu diperhatikan: aku menambah fungsi `hitungDiskon` di file bantuan bersama.

Baris "Perlu diperhatikan" sangat berguna di kerja tim. Baris itu memberi tahu *reviewer* dan pengelola repo bahwa ada file bersama yang ikut berubah, sehingga anggota lain bisa bersiap bila nanti muncul konflik.

BAB 3

Code Review dan *Merge*

PR yang sudah dibuat belum langsung digabung. Satu teman memeriksanya lebih dulu. Setelah disetujui, pengelola repo menggabungkannya ke main.

3.1 Apa Itu *Code Review*?

> **DEFINISI**
> ***Code review*** adalah pemeriksaan perubahan kode oleh orang lain sebelum perubahan itu digabung. Orang yang memeriksa disebut ***reviewer***.

Hampir semua perusahaan teknologi mewajibkan *code review* sebelum kode masuk ke versi utama. Tujuannya bukan mencari siapa yang salah, tetapi menemukan masalah selagi masih murah diperbaiki. Di aplikasi dompet digital, misalnya, *reviewer* bisa menemukan tombol transfer yang masih bisa ditekan dua kali sebelum fitur itu dipakai jutaan orang. *Review* juga membuat setidaknya dua orang memahami setiap bagian aplikasi, sehingga pekerjaan tidak bergantung pada satu orang.

3.2 Cara Meminta *Reviewer*



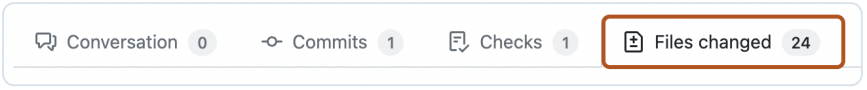
Sumber gambar: GitHub Docs (CC BY 4.0)

Kotak Reviewers berada di sisi kanan halaman PR. Dokumentasi GitHub menjelaskan dua cara memilih *reviewer*.

1. **Reviewers.** Klik judul ini, ketik nama pengguna teman, lalu pilih namanya. Hanya akun yang punya akses ke repo yang bisa dipilih.
2. **Request (kotak oranye).** Bila GitHub menampilkan saran nama, klik Request di samping nama itu.

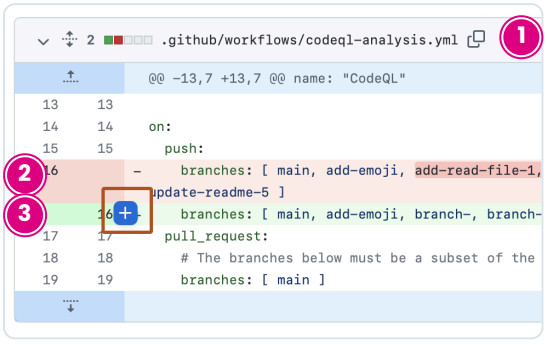
3.3 Cara Membaca Tab Files changed

Halaman PR punya beberapa tab. Conversation berisi deskripsi dan komentar, Commits berisi riwayat *commit*, dan **Files changed** (kotak oranye) berisi perubahan kode yang dibaca *reviewer*.



Sumber gambar: GitHub Docs (CC BY 4.0)

Tab Files changed menampilkan perbedaan antara main dan *branch* fitur, yang disebut *diff*. Tangkapan layar berikut memperlihatkan satu potongan *diff*.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Nama file.** Baca daftar file lebih dulu. Semua file seharusnya milik fitur itu, kecuali yang disebut di deskripsi.
2. **Baris merah dengan tanda `-`.** Baris yang dihapus.
3. **Baris hijau dengan tanda `+`.** Baris baru.
4. **Ikon `+` biru (kotak oranye).** Muncul saat kursor diarahkan ke sebuah baris. Klik untuk menulis komentar tepat di baris itu.

3.4 Contoh Komentar *Review* yang Membantu

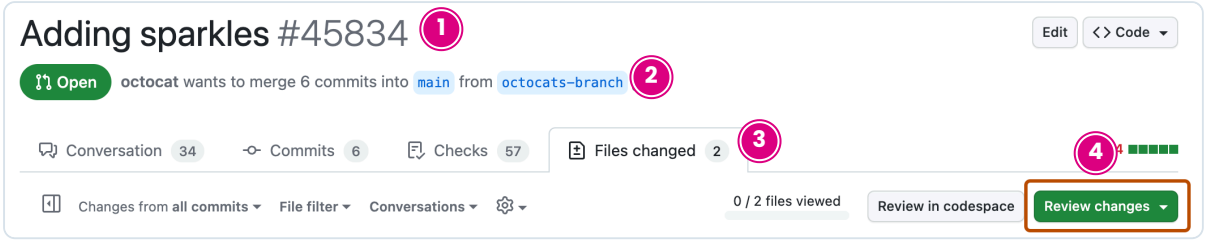
Panduan *code review* Google menyarankan komentar yang sopan, menunjuk kode dan bukan orangnya, serta menjelaskan alasannya. Komentar yang membantu menyebut apa yang dilihat, di mana, dan apa yang diharapkan. Contoh berikut berasal dari *review* fitur kode promo.

Kurang membantu	Membantu
"Ini salah."	"Kode `HEMAT10` masih diterima setelah tanggal berlakunya habis. Menurut spesifikasi seharusnya ditolak. Bisa dicek lagi?"
"Kok kamu ubah file orang?"	"File bantuan bersama ikut berubah, dan Ani juga sedang menambah fungsi di file itu. Bagaimana kalau kalian sepakati satu versi?"
"Gak jalan."	"Saat kode promo dihapus dari kolom, total tidak kembali ke harga awal."
"Oke."	"Sudah kucoba dengan kode aktif, kedaluwarsa, dan kosong. Semuanya sesuai."

Komentar persetujuan pun sebaiknya menyebut apa yang sudah dicoba, agar pengelola repo tahu PR itu benar-benar diuji.

3.5 Cara Memberi Approve atau Request Changes

Setelah memeriksa, *reviewer* mengirim keputusannya lewat tombol **Review changes** di tab Files changed. Tangkapan layar berikut juga memperlihatkan bagian atas halaman PR.



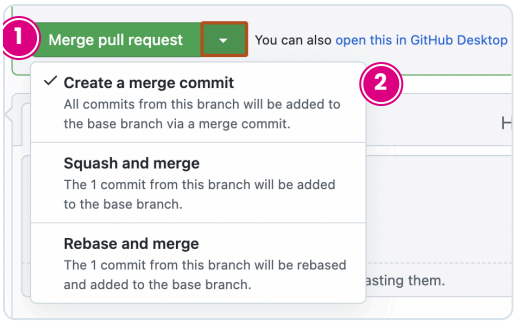
Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Judul dan nomor PR.** Nomor dipakai saat menyebut PR, misalnya "PR #2 sudah masuk main".
2. **Status dan arah.** Open berarti belum digabung. Tertulis juga *branch* tujuan (main) dan *branch* asal.
3. **Tab Files changed.** Tempat *reviewer* membaca perubahan.
4. **Review changes.** Membuka kotak keputusan *review*.

Kotak Review changes berisi kolom ringkasan dan tiga pilihan. Pilih **Comment** untuk komentar umum tanpa keputusan, **Approve** bila perubahan siap digabung, atau **Request changes** bila ada yang harus diperbaiki dulu. Klik **Submit review** untuk mengirim. Bila diminta perbaikan, pemilik fitur memperbaiki di *branch* yang sama lalu *push* lagi; *commit* baru otomatis masuk ke PR yang sama.

3.6 Apa Itu *Merge*?

> **DEFINISI**
>
> ***Merge*** adalah menggabungkan perubahan dari satu *branch* ke *branch* lain. *Merge* sebuah PR berarti fitur resmi masuk ke main.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Merge pull request.** Klik tombol ini, lalu **Confirm merge**. Tombol ini nonaktif selama PR masih punya konflik.
2. **Panah pilihan (kotak oranye).** Membuka cara penggabungan lain.
3. **Create a merge commit.** Cara bawaan: semua *commit* dari *branch* ditambahkan ke main lewat satu *commit* gabungan. Cara ini cukup untuk tim pemula.

Setelah *merge*, main berubah. Semua anggota menarik (*pull*) main terbaru ke *branch*-nya supaya perbedaan tidak menumpuk.

BAB 4

Apa Itu *Merge Conflict*?

Git biasanya bisa menggabungkan dua *branch* sendiri. Ada satu keadaan ketika Git berhenti dan meminta manusia memutuskan, yaitu ketika dua orang mengubah bagian yang sama di file yang sama.

4.1 Kapan *Merge Conflict* Terjadi?

> **DEFINISI**
> ***Merge conflict*** adalah keadaan ketika dua *branch* mengubah baris yang sama di satu file dengan isi berbeda, atau satu *branch* mengubah file yang dihapus *branch* lain, sehingga Git tidak tahu versi mana yang dipakai.

Konflik bukan tanda ada yang berbuat salah. Konflik hanya berarti dua orang mengerjakan bagian yang sama. Di tim aplikasi berita, misalnya, satu pengembang mengganti urutan menu di bilah atas, sementara pengembang lain menambah menu Video di baris yang sama. Kedua perubahan masuk akal, tetapi Git tidak bisa menebak susunan menu yang diinginkan. File yang sering bentrok adalah file yang dipakai bersama, seperti file fungsi bantuan, daftar paket (`package.json`), dan file tata letak menu.

4.2 Seperti Apa Penanda Konflik di File?

Git menandai bagian yang bentrok dengan tiga baris penanda. Contoh berikut muncul di laptop Budi (tim toko online) saat ia menarik main yang sudah berisi perubahan Ani pada teks tombol yang sama.

```
// tombol di halaman checkout
<<<<<<< HEAD                          (1)
<Button>Bayar dan Pakai Promo</Button>
=======                               (2)
<Button>Lanjut ke Pembayaran</Button>
>>>>>>> main                          (3)
```

1. `<<<<<<< HEAD`. Awal bagian bentrok. Baris sesudahnya adalah versi di *branch* yang sedang aktif, milik Budi.
2. `=======`. Pemisah antara versi Budi dan versi yang datang.
3. `>>>>>>> main`. Akhir bagian bentrok. Baris di atasnya versi dari main, milik Ani. Labelnya bisa berupa nama *branch* atau kode *commit*.
4. **Baris di luar penanda.** Sudah tergabung dengan baik dan tidak perlu disentuh.

Menyelesaikan konflik berarti menulis versi akhir yang disepakati, lalu menghapus ketiga penanda. Selama penanda masih ada, aplikasi tidak bisa di-*build*. Penyelesaiannya bukan memilih secara acak: bila bagian itu milik orang lain, putuskan bersama pemiliknya.

4.3 Tampilan Konflik di Halaman *Pull Request*

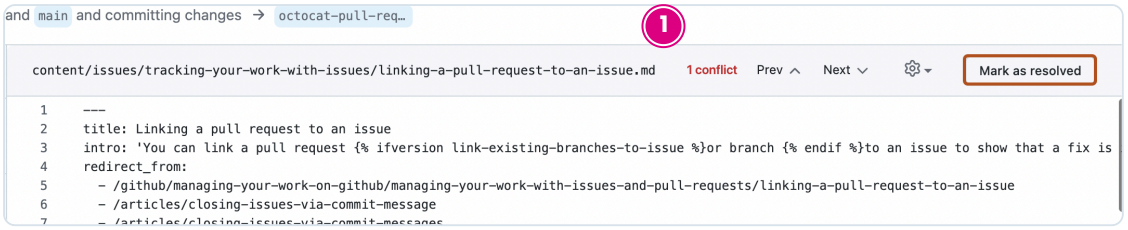
Bila konflik terjadi antara *branch* PR dan main, GitHub menampilkan peringatan di bagian bawah halaman PR. Dokumentasi GitHub menegaskan bahwa semua konflik harus diselesaikan sebelum PR bisa digabung.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Peringatan konflik.** *Branch* ini punya konflik yang harus diselesaikan.
2. **Conflicting files.** Daftar file yang bentrok. Tombol **Resolve conflicts** (kotak oranye) membuka editor konflik.

Di editor konflik, setiap file ditampilkan dengan penanda seperti di subbab 4.2. Setelah file dirapikan, klik **Mark as resolved** (kotak oranye), ulangi untuk file lain, lalu klik **Commit merge**. Editor ini hanya bisa dipakai untuk konflik baris yang sederhana; konflik lain diselesaikan di laptop.



Sumber gambar: GitHub Docs (CC BY 4.0)

1. **Jumlah konflik di file ini.** Tombol Prev dan Next di sampingnya berpindah antarkonflik di file yang sama.

4.4 Cara Mencegah *Merge Conflict*

Konflik tidak bisa dihilangkan sepenuhnya, tetapi bisa dibuat jarang dan kecil. Kebiasaan berikut paling berpengaruh.

- [ ] **Satu fitur, satu folder.** Setiap anggota hanya mengubah file milik fiturnya.
- [ ] **Kabari sebelum mengubah file bersama.** Anggota lain yang butuh perubahan serupa cukup menunggu dan memakai milikmu.
- [ ] **Tarik main setiap ada PR yang masuk.** Semakin sering menarik main, semakin kecil perbedaan yang menumpuk.
- [ ] **Kirim PR selagi kecil.** PR yang lama menunggu sambil terus bertambah lebih mungkin bentrok.

BAB 5

Komunikasi dan Pengujian dalam Tim

GitHub mengatur cara kode bertemu. Namun alat itu tidak bisa memberi tahu temanmu bahwa kamu akan mengubah database, bahwa kamu sedang terhambat, atau bahwa ada masalah di aplikasinya. Bagian itu dijalankan lewat kebiasaan kerja sama.

5.1 Kapan Anggota Tim Harus Memberi Kabar?

Tim pengembang umumnya punya beberapa saat ketika kabar wajib disampaikan, entah lewat ucapan langsung atau kanal obrolan tim.

Saat	Kenapa wajib	Contoh di tim toko online
Sebelum mengubah sesuatu yang dipakai bersama	Database dan file bersama tidak punya jalur terpisah. Perubahannya langsung terasa oleh semua.	*"Aku mau menambah kolom* `kode_promo` *di tabel pesanan, ya."*
Saat terhambat	Hambatan yang disimpan sendiri menghabiskan waktu. Teman mungkin sudah tahu jalan keluarnya.	*"Aku terhambat. Ongkir selalu nol untuk alamat luar Jawa. Ada yang bisa lihat?"*
Saat PR siap	*Reviewer* perlu tahu ada yang menunggu diperiksa.	*"Citra, PR kode promo siap. Kamu reviewer-nya."*

Kabar yang baik singkat dan spesifik. "Aku ubah database ya" tidak cukup; sebutkan apa yang diubah agar teman bisa menilai apakah pekerjaannya terdampak.

5.2 Apa Itu Cek Singkat Harian?

Banyak tim menjalankan pertemuan singkat setiap hari, sering disebut *stand-up*. Setiap anggota menjawab tiga hal secara bergiliran: apa yang sudah selesai, apa yang sedang dikerjakan, dan apa yang menghambat. Pertemuan ini bukan tempat menyelesaikan masalah. Bila ada hambatan, pemimpin pertemuan cukup menunjuk siapa yang membantu setelah pertemuan selesai, sehingga waktunya tetap singkat.

5.3 Apa Itu *Pair Testing*?

> **DEFINISI**
>
> ***Pair testing*** adalah cara menguji aplikasi berdua dalam satu waktu: satu orang memegang perangkat dan mencoba, satu orang lain mengamati dan mencatat.

Pembuat aplikasi cenderung menguji dengan cara yang sudah ia bayangkan. Orang dari luar mencoba dengan cara berbeda dan lebih mudah menemukan celah. Di aplikasi pemesanan makanan, misalnya, penguji dari tim lain cepat menemukan tombol batal yang masih aktif setelah pesanan diantar. Tabel berikut membagi tugas kedua peran.

Peran	Yang dilakukan	Yang dihindari
Penguji	Memegang perangkat, mencoba skenario uji, mengucapkan apa yang sedang dilakukan, dan menulis laporan temuan.	Diam saja saat mencoba, atau melewatkan skenario karena "pasti aman".
Pembuat	Menyiapkan aplikasi dan akun uji, mengamati tanpa mengarahkan, dan mencatat.	Mengambil alih perangkat, memberi jalan pintas, atau membela diri.

5.4 Cara Menulis Laporan Temuan

Temuan yang tidak tertulis dengan jelas sulit diperbaiki. Pembuat harus bisa mengulang langkahnya sendiri dan melihat masalah yang sama. Karena itu laporan temuan memakai lima bagian: **Akun** (akun yang dipakai), **Langkah** (urutan tindakan sampai masalah muncul), **Seharusnya** (hasil yang benar menurut spesifikasi), **Yang terjadi** (hasil yang terlihat), dan **Bukti** (tangkapan layar atau pesan *error*). Bandingkan dua laporan untuk aplikasi toko online.

SULIT DITINDAKLANJUTI

"Promo error. Tolong diperbaiki."

MUDAH DITINDAKLANJUTI

Akun: pembeli baru
Langkah: isi keranjang Rp90.000, masukkan `HEMAT10`
Seharusnya: ditolak, minimal belanja Rp100.000
Yang terjadi: potongan 10% diterapkan
Bukti: tangkapan layar halaman *checkout*

Tulis satu laporan untuk satu temuan, supaya setiap temuan bisa diperbaiki dan diuji ulang sendiri-sendiri.

5.5 Cara Menanggapi Temuan

Menerima temuan tentang aplikasi sendiri bisa terasa tidak nyaman. Padahal setiap temuan yang ditemukan teman adalah masalah yang tidak akan dialami pengguna sungguhan. Tanggapan pertama menentukan apakah penguji mau terus memberi masukan yang jujur.

Kalimat yang diucapkan	Kalimat yang dihindari
"Terima kasih, sudah ketemu."	*"Di laptopku jalan kok."*
"Boleh tunjukkan langkahnya sekali lagi?"	*"Itu bukan bug."*
"Aku catat dulu, nanti kami perbaiki dan minta kamu uji ulang."	*"Kamu salah cara pakainya."*
"Oh, berarti aturan ini belum dicek di fungsinya."	*"Itu fitur temanku, bukan punyaku."*

Setelah menerima temuan, pemilik fitur memperbaikinya dengan alur yang sama: bekerja di *branch*, *push*, membuat PR, meminta *review*, lalu digabung. Terakhir, minta penguji mencoba ulang langkah yang sama.

BAGIAN B · MEMBANGUN APP 4 BERSAMA TIM

BAB 6

Menyiapkan Repo Tim Bengkel

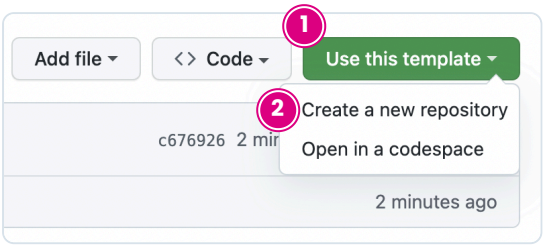
Mulai bab ini, semua cara kerja di Bagian A diterapkan pada aplikasi Bengkel Cemerlang Jaya Motor. Kamu bekerja dalam tim empat orang. Selama ini kamu hanya memakai *clone*, *commit*, dan *push* sendirian lewat agen di Antigravity; hari ini empat orang mengubah satu repo dan satu database bersamaan.

6.1 Apa Isi Repo *Template* App 4?

> **DEFINISI**
> **Repo *template*** adalah repo GitHub yang ditandai sebagai cetakan. Repo baru yang dibuat darinya punya struktur folder dan file yang sama persis, tetapi riwayat *commit* bersih dan pemilik sendiri. Berbeda dengan *fork*, repo dari *template* tidak terhubung lagi dengan repo asalnya.

Repo starter App 4 adalah repo *template*. Isinya sudah mencakup login, *route guard*, sidebar per peran, semua tabel dan RLS di `supabase/migrations/01_dasar.sql`, data contoh di `seed.sql`, serta halaman Pelanggan dan Tim Mekanik. Halaman empat fitur masih kerangka bertuliskan "Halaman ini belum dibangun". Repo tim yang dibuat dari *template* ini menjadi milik tim sepenuhnya, terpisah dari repo mentor dan repo tim lain.

6.2 Cara Ketua Tim Membuat Repo dari *Template*



Sumber gambar: GitHub Docs (CC BY 4.0)

Langkah ini hanya dikerjakan ketua tim. Buka repo starter dari tautan trainer.

1. **Use this template.** Tombol hijau di atas daftar file.
2. **Create a new repository.** Pilih ini. Pilihan Open in a codespace tidak dipakai di kelas.

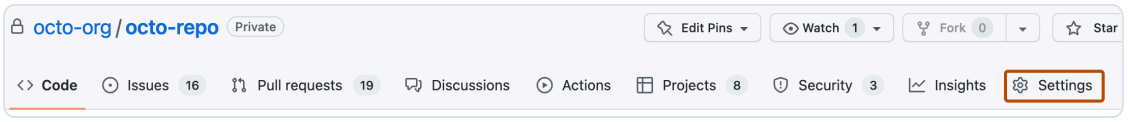
Formulir pembuatan repo kemudian terbuka. Isi sesuai langkah berikut.

1. **Owner:** pilih akun GitHub ketua tim.
2. **Repository name:** isi `cemerlang-jaya-k1`, ganti angka 1 dengan nomor tim.
3. **Visibility:** pilih sesuai arahan trainer. Biarkan **Include all branches** tidak dicentang, karena yang dibutuhkan hanya main.
4. Klik **Create repository from template**. Halaman repo tim terbuka.

6.3 Cara Mengundang Anggota sebagai *Collaborator*

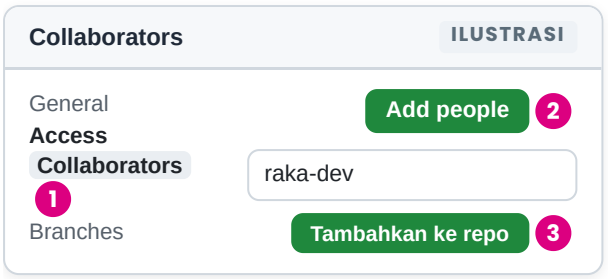
> **DEFINISI**
> ***Collaborator*** adalah akun GitHub yang diundang pemilik repo pribadi untuk ikut bekerja di repo itu. Setelah undangan diterima, *collaborator* bisa mengirim *branch*, membuat *pull request*, dan mereview.

Kumpulkan dulu nama pengguna GitHub ketiga anggota tim. Di repo tim, buka tab **Settings** (kotak oranye pada tangkapan layar berikut).



Sumber gambar: GitHub Docs (CC BY 4.0)

Di menu samping kiri bagian **Access**, klik **Collaborators**, lalu **Add people**. Ilustrasi berikut memperlihatkan urutannya.



1. **Collaborators.** Ada di bagian Access pada menu samping.
2. **Add people.** Membuka kolom pencarian akun.
3. **Tombol konfirmasi.** Ketik nama pengguna, pilih akun yang benar, lalu klik tombol tambah. Ulangi untuk ketiga anggota. Periksa ejaan dengan teliti; undangan yang salah alamat bisa diterima orang lain.

Di waktu yang sama, ketua tim membuat proyek Supabase `cemerlang-jaya-k1` dan mengundang ketiga anggota ke organisasi Supabase sebagai Administrator. Undangan Supabase terpisah dari undangan GitHub.

6.4 Cara Anggota Tim Menerima Undangan

1. Buka email yang terhubung dengan akun GitHub, buka tautan undangan repo `cemerlang-jaya-k1`, lalu terima undangan di halaman yang terbuka. Bila email tidak ada, minta ketua tim memeriksa nama pengguna yang diundang.
2. Terima juga undangan Supabase agar agen di laptopmu bisa membuka proyek tim lewat Supabase MCP.
3. Minta agen meng-*clone* repo tim ke Local Disk D, lalu buka foldernya di Antigravity.
4. Tunggu ketua tim menyampaikan "Database siap", lalu kirim Prompt 0 dari Daftar Prompt.

> **TANDA BERHASIL**
> Repo tim tampil di daftar repo akunmu, folder repo terbuka di Antigravity, dan `localhost:3000` bisa dibuka dengan akun Pak Darto.

6.5 Pembagian Fitur A sampai D

Contoh di bagian ini memakai satu tim berisi Dina, Raka, Sari, dan Bima. Dina menjadi ketua tim.

Anggota	Fitur	*Branch*	Folder halaman yang dipegang
Dina (ketua)	A. *Work Order*	`fitur/work-order`	`work-orders`, `work-orders/baru`, `work-orders/[id]`
Raka	B. Tagihan dan *Invoice*	`fitur/tagihan`	`work-orders/[id]/tagihan`, `work-orders/[id]/invoice`
Sari	C. *Inventory* dan Pengguna	`fitur/inventory`	`inventory`, `pengguna`, `setelan`
Bima	D. *Dashboard* dan *Audit Log*	`fitur/dashboard`	`dashboard`, `audit-log`, `laporan-laba`

Ketua tim tetap mengerjakan satu fitur. Tugas tambahannya: menyiapkan repo dan proyek Supabase, memimpin cek singkat, menggabungkan PR ke main, dan menayangkan aplikasi ke Netlify.

BAB 7

Mengerjakan dan Menggabungkan Fitur Bengkel

Bab ini mengikuti satu fitur, yaitu Fitur B milik Raka, dari *branch* sampai masuk ke main. Semua langkah Git dikerjakan agen lewat prompt dari Daftar Prompt; kamu tidak perlu mengetik perintah Git.

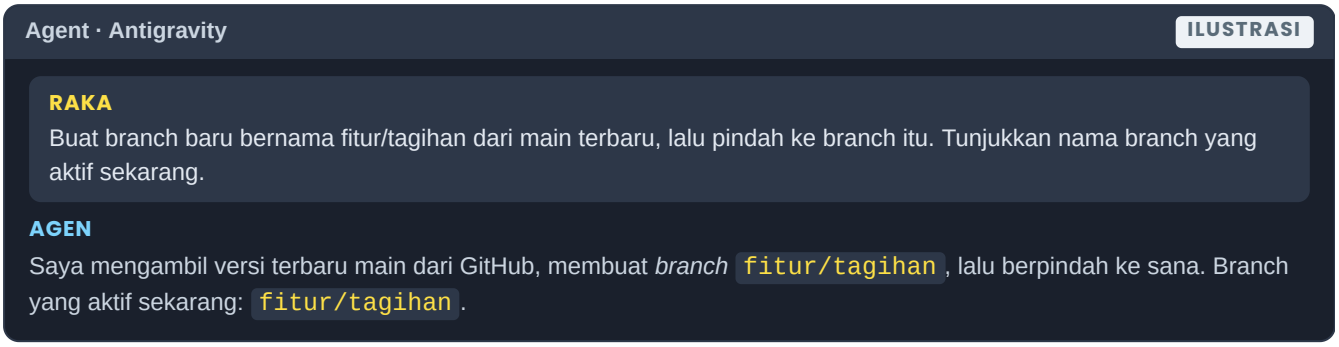
7.1 Cara Membuat *Branch* Fitur lewat Agen

Kirim Prompt *Branch*. Teksnya sama dengan Daftar Prompt; ganti `fitur/tagihan` dengan *branch* fiturmu.

PROMPT BRANCH · DAFTAR PROMPT BAGIAN 2

Buat branch baru bernama fitur/tagihan dari main terbaru, lalu pindah ke branch itu. Tunjukkan nama branch yang aktif sekarang.

Permintaan terakhir, menyebutkan *branch* aktif, menjadi bukti bahwa langkahnya berhasil. Ilustrasi percakapan berikut memperlihatkan jawaban agen di laptop Raka.



Biasakan memeriksa *branch* aktif sebelum mengirim prompt fitur, terutama setelah mencoba fitur teman. Tanyakan ke agen "Branch apa yang sedang aktif?", atau lihat bilah bawah editor; editor turunan VS Code umumnya menampilkan nama *branch* di pojok kiri bawah.

PERHATIAN

Bila agen menyebut `main` sebagai *branch* aktif saat kamu akan mengerjakan fitur, berhenti dulu dan kirim ulang Prompt *Branch*.

7.2 File Mana yang Boleh Diubah?

Starter disusun supaya keempat fitur hampir tidak perlu menyentuh file yang sama. Contohnya, halaman rincian WO sudah punya tab "Rincian | Tagihan" yang menautkan ke folder `tagihan`, sehingga Fitur A dan Fitur B tidak perlu mengubah file yang sama.

Jenis file	Contoh	Aturan
Milik satu fitur	`src/app/(app)/inventory/`, `supabase/migrations/fitur_c_inventory.sql`	Bebas diubah pemilik fitur.
File bersama	`src/app/(app)/layout.tsx` (sidebar), `src/lib/peran.ts`, `src/lib/utils.ts`, `globals.css`, `package.json`	Sampaikan ke tim sebelum mengubah. Putuskan siapa yang mengubah.
Dibuat ulang agen	`src/types/database.ts`	Tidak disunting manual.

Tidak boleh diubah	`supabase/migrations/01_dasar.sql`, `docs/PRD.md`, `AGENTS.md`	Tolak bila agen mengusulkan perubahan.

Periksa daftar file yang diubah agen setiap selesai satu prompt. Bila ada file di luar folder fiturmu, tanyakan alasannya atau minta agen membatalkannya.

7.3 Aturan Database Bersama

Keempat anggota memakai satu proyek Supabase. **Kode punya *branch*, database tidak**: SQL yang kamu setujui langsung berlaku untuk seluruh tim. Bayangkan agen Sari mengusulkan kolom baru di tabel `items` dan Sari langsung menyetujuinya, sementara agen Raka sedang menulis fungsi tagihan yang membaca tabel itu. Fungsi Raka dibuat berdasarkan struktur lama. Karena itu tim memakai empat aturan.

- [ ] Sampaikan ke tim sebelum menyetujui SQL dari agen, misalnya "Aku mau menyetujui fungsi tagihan, ya."
- [ ] Jangan membuat, mengubah, atau menghapus tabel. Tabel sudah lengkap di starter.
- [ ] Minta agen menyimpan SQL yang berhasil ke `supabase/migrations/fitur_<nama>.sql` supaya ikut terlihat di PR.
- [ ] Setelah database berubah, buat ulang `src/types/database.ts` lewat agen.

Nama fungsi, kolom, dan status juga wajib mengikuti PRD. Tombol Serahkan Mobil di Fitur A, misalnya, hanya aktif bila *invoice* dari Fitur B berstatus `lunas`. Bila Fitur A memeriksa nilai lain, tombol itu tidak pernah aktif walaupun kedua fitur berjalan baik sendiri-sendiri.

7.4 Cara Mengirim Fitur dengan Prompt *Push*

Setelah semua uji fiturnya lolos, Raka mengirim Prompt *Push*.

PROMPT PUSH · DAFTAR PROMPT BAGIAN 8

Tarik main terbaru ke branch-ku. Bila ada merge conflict, berhenti dan tunjukkan dulu. Bila aman, commit semua perubahan dengan pesan yang menjelaskan isinya, lalu push branch-ku ke GitHub. Tuliskan juga ringkasan perubahan dan cara mengujinya untuk deskripsi pull request.

Agen menarik main terbaru lebih dulu supaya *branch* Raka memuat fitur teman yang lebih dulu masuk. Bila ada bentrokan, agen berhenti dan menunjukkannya (subbab 7.8). Bila aman, agen melakukan *commit* dan *push*, lalu menulis ringkasan. Raka membuka repo tim, klik **Compare & pull request**, menempel ringkasan ke deskripsi, memilih Sari sebagai *reviewer*, lalu klik **Create pull request**. Deskripsinya kira-kira seperti ini.

CONTOH DESKRIPSI PR FITUR B

Judul: Fitur B: halaman tagihan, *invoice*, dan pembayaran
Yang dibuat: enam fungsi tagihan (PRD 9.2), halaman `/work-orders/[id]/tagihan` dan `/invoice`. SQL di `supabase/migrations/fitur_b_tagihan.sql`.
Cara menguji: masuk sebagai Rina, buka WO siap diambil, Terbitkan *Invoice*, catat pembayaran sebagian lalu lunas. Masuk sebagai Pak Joko: tombol Terbitkan *Invoice* tidak tampil.
Perlu diperhatikan: aku menambah `formatRupiah` di `src/lib/utils.ts`.

7.5 Cara Mencoba Fitur Teman dengan Prompt Coba

Sebagai *reviewer*, Sari membaca tab Files changed, lalu mencoba fiturnya di laptop sendiri dengan Prompt Coba.

PROMPT COBA · DAFTAR PROMPT BAGIAN 8

Simpan dulu pekerjaanku dengan commit. Lalu ambil branch fitur/tagihan dari GitHub dan pindah ke branch itu supaya aku bisa mencoba fitur temanku. Setelah selesai, kembalikan aku ke branch-ku sendiri.

Karena database dipakai bersama, fungsi milik Raka sudah terpasang di Supabase, jadi halaman tagihan bisa langsung dicoba di `localhost:3000`. Selama berada di *branch* teman, jangan meminta agen mengubah kode; catat temuan sebagai komentar di PR. Setelah selesai, pastikan agen sudah mengembalikanmu ke *branch*-mu sendiri.

7.6 Empat Hal yang Diperiksa *Reviewer*

No	Yang diperiksa	Cara memeriksa
1	**Deskripsi jelas**	Menyebut apa yang dibuat dan cara mengujinya.
2	**Fitur berjalan**	Jalankan langkah uji dengan Prompt Coba. Coba juga akun yang seharusnya ditolak, misalnya Pak Joko membuka halaman khusus Admin.
3	**Tidak menyentuh fitur lain**	Files changed hanya berisi folder fitur itu, file SQL fiturnya, dan file bersama yang disebut di deskripsi.
4	**Tidak ada rahasia**	Tidak ada `.env.local`, *secret key* Supabase, atau kata sandi selain akun uji kelas.

Contoh komentar yang membantu: "Tombol Terbitkan *Invoice* masih muncul untuk Pak Joko. Menurut PRD 6.9 hanya Admin dan Owner. Bisa dicek lagi?" Bila keempat hal terpenuhi, Sari memilih Approve dan menulis akun apa saja yang sudah ia coba.

7.7 *Merge* oleh Ketua dan Prompt Tarik Main

Hanya ketua tim yang menekan tombol *merge*, supaya main berubah satu per satu dan selalu diketahui satu orang. Dina memastikan PR sudah punya satu Approve dan tidak ada tanda konflik, klik **Merge pull request** lalu **Confirm merge**, kemudian menyampaikan langsung ke tim: "PR fitur B sudah masuk main." Setelah mendengar kabar itu, semua anggota mengirim Prompt Tarik Main.

PROMPT TARIK MAIN · DAFTAR PROMPT BAGIAN 8

Tarik main terbaru dari GitHub ke branch-ku. Bila ada merge conflict, berhenti dan tunjukkan dulu.

7.8 Contoh Konflik di `src/lib/utils.ts`

Bima (Fitur D) juga butuh fungsi rupiah untuk *dashboard*, dan agennya menambah `formatRupiah` di file yang sama dengan Raka. Saat Bima mengirim Prompt Tarik Main setelah PR Raka masuk, agen berhenti dan menunjukkan isi file berikut.

```
// src/lib/utils.ts
<<<<<<< HEAD                                  versi Bima
export function formatRupiah(nilai: number) {
  return new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR",
maximumFractionDigits: 0 }).format(nilai);
}
=======
export function formatRupiah(angka: number) {
  return "Rp" + angka.toLocaleString("id-ID");
}
>>>>>>> main                                  versi Raka
```

Jangan meminta agen "pokoknya selesaikan", karena agen bisa memilih satu versi dan menghapus pekerjaan teman. Kirim Prompt Konflik, yang meminta agen menjelaskan dulu dan menunggu persetujuan.

PROMPT KONFLIK · DAFTAR PROMPT BAGIAN 8

Ada merge conflict. Untuk setiap file yang bentrok:
1. Tunjukkan bagian yang bentrok dan jelaskan isi kedua versinya dengan bahasa sederhana.
2. Usulkan penyelesaian yang mempertahankan pekerjaan kedua fitur.
3. Tunggu persetujuanku sebelum mengubah file.
Khusus src/types/database.ts, jangan digabung manual. Buat ulang lewat generate_typescript_types.
Setelah selesai, jalankan npm run build untuk memastikan tidak ada error.

1. Baca penjelasan kedua versi dan tentukan file itu milik siapa. `utils.ts` adalah file bersama, jadi Bima mendatangi Raka untuk menyepakati satu versi.
2. Setujui usulan agen yang mempertahankan pekerjaan kedua fitur, misalnya satu fungsi yang dipakai halaman tagihan dan *dashboard*.
3. Tunggu `npm run build` berhasil, lalu kirim Prompt *Push* lagi.

TANDA BERHASIL

Tidak ada lagi penanda `<<<<<<<`, `=======`, atau `>>>>>>>` di file, `npm run build` berhasil, dan halaman kedua fitur tetap berjalan.

7.9 Konflik di `src/types/database.ts`

File ini berisi daftar tabel, kolom, dan fungsi database dalam bentuk tipe TypeScript, dan dibuat agen lewat `generate_typescript_types`. Setiap anggota membuatnya ulang setelah fungsinya terpasang, jadi setiap *branch* punya versi berbeda dan konflik hampir pasti terjadi. Jangan menggabungnya baris demi baris atau memilih salah satu versi: versi mana pun bisa kehilangan fungsi milik fitur lain. Database bersama sudah berisi semua fungsi keempat fitur, jadi cukup buat ulang file dari database. Prompt Konflik sudah memuat aturan ini. Bila editor menandai nama fungsi sebagai *error*, kirim Prompt Memperbarui Tipe dari Daftar Prompt.

BAB 8

Kabar, Cek Singkat, dan Uji Silang Antartim

Kebiasaan di Bab 5 dijalankan di kelas dengan aturan sederhana. Tim duduk berdekatan, jadi kabar disampaikan langsung dengan suara, bukan lewat grup WhatsApp.

8.1 Tiga Saat Wajib Memberi Kabar di Kelas

Saat	Contoh kalimat di tim bengkel
Sebelum mengubah database	*"Aku mau menyetujui fungsi* `catat_stok_masuk` *dan* `sesuaikan_stok`*. Tabel tidak diubah."*
Saat terhambat	*"Aku terhambat. Halaman inventory kosong untuk Rina, padahal datanya ada. Sudah kucek RLS-nya. Ada yang bisa lihat sebentar?"*
Saat PR siap	*"Sari, PR fitur B sudah siap. Aku memilihmu sebagai reviewer."*

Ketua tim juga menyampaikan kabar setiap kali main berubah, misalnya "PR fitur A sudah masuk main". Kabar ini menjadi tanda bagi semua anggota untuk mengirim Prompt Tarik Main.

8.2 Cek Singkat Pukul 11.30 dan 13.45

Pada pukul 11.30 dan 13.45, ketua tim memimpin cek singkat tiga menit. Setiap anggota menyebut apa yang sudah, apa yang sedang dikerjakan, dan apa yang menghambat. Contoh cek singkat tim Dina pukul 13.45:

Anggota	Sudah	Sedang	Terhambat
Dina	Fungsi WO, halaman daftar WO	Rincian WO dan foto	Tidak ada
Raka	Fungsi tagihan, halaman tagihan	Halaman *invoice*	Tidak ada
Sari	Fungsi stok	Halaman *inventory*	Harga modal tampil untuk Rina
Bima	*Audit log*, laporan laba	*Dashboard*	Menunggu `formatRupiah` dari PR Raka

Dari contoh ini, ketua tim cukup meminta Sari memanggil mentor soal harga modal, dan meminta Raka mengirim PR lebih dulu agar Bima tidak menulis ulang fungsi yang sama.

8.3 Skenario Uji Silang Antartim

Pukul 15.15, tim dipasangkan. Dua orang dari timmu menguji aplikasi tim pasangan selama 15 menit dengan peran penguji, dua orang lainnya tetap sebagai pembuat (subbab 5.3). Skenario berikut dari Petunjuk Pengerjaan menguji aturan akses dan aturan bisnis yang paling penting.

No	Skenario	Akun	Hasil yang benar
1	Buka WO milik Tim B dan coba ubah statusnya	Pak Joko	Tidak bisa
2	Ketik `/audit-log` dan `/laporan-laba` di alamat	Rina	Dialihkan
3	Cari harga modal di *Inventory* dan DevTools › Network	Rina	Tidak ada

4	Tambah stok terakhir dari dua tab bersamaan	Pak Joko	Satu ditolak
5	Serahkan mobil sebelum *invoice* lunas	Rina	Ditolak

Catatan: "Tim B" di skenario 1 adalah tim mekanik Pak Slamet di dalam aplikasi, bukan tim peserta.

8.4 Contoh Laporan Temuan Aplikasi Bengkel

Laporan ditulis di lembar temuan dengan lima bagian dari subbab 5.4, lalu diserahkan ke tim pemilik aplikasi.

SULIT DITINDAKLANJUTI

"Status WO bisa diubah sembarangan. Tolong diperbaiki."

MUDAH DITINDAKLANJUTI

Akun: Pak Joko (Tim A)
Langkah: Buka WO-2610-0011 milik Tim B, klik Mulai Kerjakan
Seharusnya: Tombol tidak tampil atau ditolak
Yang terjadi: Status berubah menjadi dikerjakan
Bukti: Tangkapan layar halaman WO

Tim yang menerima laporan menanggapinya seperti di subbab 5.5: berterima kasih, meminta ditunjukkan langkahnya, lalu pemilik fitur memperbaikinya lewat *branch* dan PR baru dan meminta penguji mencoba ulang.

Daftar Istilah

Istilah	Arti
Branch	Jalur kerja terpisah di dalam satu repo. Perubahan di sebuah *branch* tidak memengaruhi *branch* lain sampai digabungkan.
Code review	Pemeriksaan perubahan kode oleh orang lain sebelum perubahan itu digabung ke main.
Collaborator	Akun GitHub yang diundang pemilik repo pribadi agar bisa mengirim *branch*, membuat *pull request*, dan mereview.
Commit	Simpanan keadaan kode pada satu titik waktu beserta pesan yang menjelaskan perubahannya.
Diff	Tampilan perbedaan dua versi file: baris baru hijau, baris yang dihapus merah.
main	*Branch* utama repo yang berisi versi aplikasi yang sudah disetujui.
Merge	Penggabungan perubahan dari satu *branch* ke *branch* lain. Di tim bengkel dilakukan oleh ketua tim.
Merge conflict	Keadaan ketika dua *branch* mengubah bagian yang sama di satu file dengan isi berbeda, sehingga manusia harus memutuskan versi akhirnya.
Pair testing	Pengujian berdua: satu orang mencoba aplikasi, satu orang mengamati dan mencatat.
Pull	Mengambil *commit* terbaru dari GitHub lalu menggabungkannya ke *branch* di laptop.
***Pull request* (PR)**	Usulan untuk menggabungkan perubahan dari satu *branch* ke *branch* lain, lengkap dengan deskripsi, *diff*, dan hasil *review*.
Push	Mengirim *commit* dari laptop ke repo di GitHub.
Repo *template*	Repo yang ditandai sebagai cetakan untuk membuat repo baru dengan isi sama dan riwayat bersih.
Reviewer	Orang yang diminta memeriksa *pull request* dan memberi keputusan Approve, Request changes, atau Comment.

Daftar Pustaka

Chacon, S., & Straub, B. (2014). *Pro Git* (edisi ke-2), bab 3.2 *Basic Branching and Merging*. git-scm.com. https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

Git Project. (t.t.). *git-merge Documentation* (bagian *How Conflicts Are Presented*). git-scm.com. https://git-scm.com/docs/git-merge

GitHub. (t.t.). *About branches* dan *GitHub flow*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches dan https://docs.github.com/en/get-started/using-github/github-flow

GitHub. (t.t.). *About merge conflicts* dan *Resolving a merge conflict on GitHub*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts dan https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github

GitHub. (t.t.). *About pull requests*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

GitHub. (t.t.). *Creating a pull request* dan *Requesting a pull request review*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request dan https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/requesting-a-pull-request-review

GitHub. (t.t.). *Creating a repository from a template*. GitHub Docs. https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template

GitHub. (t.t.). *github/docs repository*, lisensi CC BY 4.0. Sumber tangkapan layar di modul ini. https://github.com/github/docs

GitHub. (t.t.). *Inviting collaborators to a personal repository*. GitHub Docs. https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-access-to-your-personal-repositories/inviting-collaborators-to-a-personal-repository

GitHub. (t.t.). *Merging a pull request*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/merging-a-pull-request

GitHub. (t.t.). *Reviewing proposed changes in a pull request*. GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/reviewing-proposed-changes-in-a-pull-request

Google. (t.t.). *How to write code review comments*. Google Engineering Practices Documentation. https://google.github.io/eng-practices/review/reviewer/comments.html